

实践6：基于教师-学生蒸馏的全身运动跟踪



深蓝学院《人形机器人运动控制》课程第4章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本作业在 **mjlab** 框架中，基于 **Unitree G1** 完成 **general motion tracker** 的 **teacher-student distillation** 训练。你的核心任务是：在已提供的 privileged teacher policy 监督下，实现并比较两种 student distillation 方法——**action matching** 与 **KL matching**。

本作业不要求：

- 重新训练 teacher policy（为减少算力需求，聚焦蒸馏算法本身，我们提供预训练权重）
- 实现 motion tracking 的完整 MDP 设计（这将会在第6章更详细的介绍）
- 实现 human motion → robot motion 的 **retargeting** 流程（已提供重定向并筛选过的数据集）

本作业要求：

- 理解 teacher 与 student 的 observation 差异
- 在 PPO 之上实现两种 distillation loss
- 训练 student 并比较两种方法的行为差异

完成本作业后，你应当能够：

能力项	说明
理解 privileged teacher	理解 teacher 为何使用 future reference motion等特权观测，以及为何不能直接部署
实现 distillation loss	完成 7 个 TODO（3 个文件）
运行完整训练流程	训练 student、Viser 回放、TensorBoard 对比
撰写简要实验报告	对比 action matching 与 KL matching 的训练与行为差异

2. 作业背景（理论）

本节介绍本作业的问题背景。请先通读本节，再阅读 TODO 实现要求——理论中的每个概念，都会在后续代码中有直接对应。

2.1 从 Reward-Based Locomotion 到 Motion Tracking

在之前的课程中，我们已经学习了 **pure reward-based locomotion**：通过速度跟踪、姿态约束、能耗正则、接触惩罚等 reward 项，手工塑造机器人的行走行为。你在 **实践2（HW2）** 中应该已经体会到：这种方法可以训练出可用的 locomotion，但 **natural gait（自然步态）** 往往较难调。reward 权重、std、课程学习等都需要反复试验。

Motion tracking 提供了一条不同的思路：

- 不再完全依赖“像人一样走”的手工 reward 设计
- 改为给策略一段 **参考运动（reference motion）**
- 让机器人尽量跟踪这段参考，从而获得更接近人类数据的步态

本作业聚焦 **locomotion via motion tracking**：在平地上跟踪来自人类行走数据的参考动作；目标是体验 motion tracking 及其如何在PPO上实现 distillation 这一重要方法。

2.2 课程脉络：三种 Locomotion 思路

范式	核心思想	优点	局限
Reward-based	用 reward 塑造 gait	不依赖参考动作，任务灵活	自然步态需要大量调试
Motion tracking	跟踪参考运动	步态自然，数据驱动	泛化能力欠佳，参考运动通常未考虑真实地形
Style-prior based	用 motion prior（如 AMP）约束风格	可结合 RL 与数据先验	下一章将介绍

本作业属于第二类。下一章课程将介绍 **style-prior based locomotion**（如 AMP 等），进一步探索如何在 RL 中引入人类运动先验。

2.3 数据来源：AMASS 与 ACCAD 子集

AMASS 是什么？

AMASS（Archive of Motion Capture As Surface Shapes）是一个大规模人体运动数据库。它将多个 optical marker-based motion capture 数据集统一到 **SMPL** 等共同人体模型参数化下，可用于动画、可视化以及深度学习训练数据生成。

为何本作业只用 ACCAD 中的 25 个 walk clips?

AMASS 规模很大，直接使用完整语料会显著增加训练与调试成本。为降低计算开销、聚焦核心概念，本作业：

- 1. 从 AMASS 所包含的 **ACCAD** 数据集中，选取 **25 个 walk clips**
- 2. 聚焦 **human-like locomotion** 这一明确子问题
- 3. 用 **一个 teacher policy** 联合跟踪全部 25 段动作，而非为每段动作单独训练一个模型

这 25 个 clips 的规模足够体现 “**一个 policy 跟踪多个 motion clips**” 的 general motion tracking 思想，同时又足够小，适合作业训练、测试与对比实验。

Retargeting 边界（重要）

上述人类 motion clips 已经完成 **retargeting** 到 **Unitree G1** 的处理，并以 motion command 的形式提供给仿真环境。**你无需实现 retargeting**。关于 motion tracking 的完整 MDP 设计、retargeting 的优化目标、接触处理、人体-机器人比例差异等细节，将在后续课程中详细介绍；本作业只关注 **teacher-student distillation** 的实现。

2.4 本作业任务：General Motion Tracker

General motion tracker 的目标是：给定一段参考运动，机器人输出关节级控制，使根部运动、身体姿态和关节轨迹尽量跟随参考。

环节	输入	输出
Motion command	当前参考帧（student）或当前+未来参考帧（teacher）	根部/关节/anchor 等跟踪目标
Policy	command + proprioception + history	关节位置
仿真	action	机器人运动，reward / termination

代码对应：

- 环境配置： `src/humanoid_hw6/teacher_env_cfg.py` 、
`src/humanoid_hw6/student_env_cfg.py`
- Motion command： `src/humanoid_hw6/mdp/motion/`
- RL 配置： `src/humanoid_hw6/config/g1/rl_cfg.py`

开始实现 **TODO** 前，请先完成 § 3 环境配置。

3. 环境配置

本节结构参考课程 **实践4（HW4）** 的环境安装指南（ `shenlan_doc/实践4：蹲姿行走策略，G1速度+骨盆高度MDP设计.pdf` §3），并按 HW6 仓库实际路径与命令做了适配。

3.1 系统要求

项目	要求
操作系统	Linux（Ubuntu 20.04+ 推荐；WSL2 可用但 Viser 需额外配置）
GPU	NVIDIA GPU（ 训练必需 ；建议 RTX 3060 及以上）
显卡驱动	支持 CUDA 12.x
磁盘空间	≥ 15 GB 可用（依赖 + 训练 checkpoint）
网络	首次安装需联网下载 Python 包
Python	3.10–3.13（由 <code>uv</code> 自动管理，无需系统预装）

可选检查 GPU 与驱动：

代码块

```
1  nvidia-smi
```

3.2 获取代码与目录结构

从课程平台获取 **实践6** 作业包后，在终端进入仓库根目录 `shenlan_humanoid_hw6/`。关键结构如下：

代码块

```
1  shenlan_humanoid_hw6/
2  |— pyproject.toml / uv.lock
3  |— assets/motions/g1_accad_walk/    # 25 个 ACCAD walk NPZ（作业已包含）
4  |— checkpoints/g1_hw6_teacher/      # 预训练 teacher checkpoint
5  |— src/humanoid_hw6/                # 学生 TODO 与 MDP 实现
6  |— tests/
```

作业 zip 不含 `.venv/`、`logs/`、`wandb/` 等本地生成内容；首次使用请执行 §3.4 的 `uv sync --dev` 安装依赖。motion 数据已随作业包提供，无需额外下载 AMASS / OMOMO 大数据集。

后续所有 `uv run ...` 命令均在 `shenlan_humanoid_hw6/` 目录下执行。

3.3 安装 uv 包管理器

实践6 使用 `uv` 管理 Python 版本与依赖。若尚未安装：

代码块

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
2 source $HOME/.local/bin/env # 或重新打开终端
3 uv --version
```

3.4 安装依赖

实践6 是独立 Python 包（依赖 PyPI 上的 `mjlab==1.2.0`）

代码块

```
1 cd shenlan_humanoid_hw6
2 uv sync --dev
```

说明：

- Linux x86_64 下，lockfile 会解析带 CUDA 12 的 PyTorch wheel
- 首次安装约需 3–8 GB 下载，视网络情况需 10–30 分钟

常见安装问题：

现象	处理
<code>uv: command not found</code>	完成 § 3.3，确认 <code>\$HOME/.local/bin</code> 在 <code>PATH</code> 中
下载 PyTorch 超时	检查网络；可多次重试, 换镜像源
CUDA / 驱动不匹配	升级 NVIDIA 驱动；或联系助教确认 GPU 型号
磁盘空间不足	清理旧 <code>.venv</code> ；确保 ≥ 15 GB 可用

3.5 快速自检

安装完成后，在 `shenlan_humanoid_hw6/` 目录下依次执行：

Step 1 — 列出已注册任务

代码块

```
1 uv run python -m mjlab.scripts.list_envs | rg Humanoid-HW6
```

预期输出包含：

代码块

```
1 Mjlab-Humanoid-HW6-Teacher-G1
2 Mjlab-Humanoid-HW6-Student-Action-Matching-G1
3 Mjlab-Humanoid-HW6-Student-KL-Matching-G1
```

Step 2 — 运行 baseline 测试

代码块

```
1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run python -m pytest
  tests/test_baseline.py -q
```

Step 3 — 确认 GPU 可用（训练机器）

代码块

```
1 uv run python -c "import torch; print('cuda:', torch.cuda.is_available())"
```

GPU 机器上应输出 `cuda: True`。

Step 4 — 确认 TODO 可定位

代码块

```
1 grep -rn "【作业 TODO" src/humanoid_hw6/rl/algorithms/ | head
```

环境 OK 的标志：三个 实践6 task ID 可见、baseline 测试通过、能搜到 7 处 TODO 标记。

3.6 GPU 与显存建议

默认 student 训练使用 4096 个并行环境。若显存不足，可降低 `--env.scene.num-envs`：

GPU 显存	建议 <code>num-envs</code>
≥ 24 GB（如 RTX 4090）	4096
16 GB	2048
12 GB	1024（训练较慢）

示例：

代码块

```
1 WANDB_MODE=disabled uv run python -m mjlab.scripts.train \  
2   Mjlab-Humanoid-HW6-Student-Action-Matching-G1 \  
3   --env.scene.num-envs 4096 \  
4   ...
```

3.7 预览 25 个 ACCAD walk 参考动作 (Viser)

安装完成后，建议先用 Viser 浏览参考 **motion**，确认 retarget 后的 G1 轨迹，再开始实现 distillation TODO。

用途： 不运行 policy，在浏览器中逐条播放 `assets/motions/g1_accad_walk/*.npz`。

命令（浏览全部 25 clips）：

代码块

```
1 cd shenlan_humanoid_hw6  
2 uv run python -m humanoid_hw6.scripts.data.visualize_motion_curate_viser \  
3   --motion assets/motions/g1_accad_walk
```

也可省略 `--motion` （默认即 `assets/motions/g1_accad_walk`）：

代码块

```
1 uv run python -m humanoid_hw6.scripts.data.visualize_motion_curate_viser
```

单 clip 调试：

代码块

```
1 uv run python -m humanoid_hw6.scripts.data.visualize_motion_curate_viser \  
2   --motion assets/motions/g1_accad_walk/accad_B3___walk1.npz
```

操作说明：

- 终端会打印 Viser URL，在浏览器中打开
- 使用 clip 切换控件浏览 25 段动作
- 播放 / 暂停、调整播放速度
- **本作业不要求** keep/reject 标注；`--state-json` 可省略

4. 教师策略简介

虽然本作业 **不要求你训练 teacher**，但理解 teacher 为何有效，是理解 distillation 的前提。

4.1 为何需要 Privileged Teacher?

Teacher policy 在训练时可以使用 **future reference motion**，即“当前时刻 + 未来若干帧”的参考运动。这是一种 **privileged information/**.

本作业提供 **一个共享 teacher**，联合跟踪全部 **25 个 ACCAD walk clips**，而不是 25 个独立模型。默认 checkpoint 路径：

代码块

```
1 checkpoints/g1_hw6_teacher/model_latest.pt
```

4.2 Teacher Observation Space（3007 维）

Teacher actor 的 observation 由三部分组成：

模块	维度	含义
Command	1344	21 个参考步 × 64 维特征（ <code>t+0</code> 当前帧 + 20 个 future offsets）
Proprio	93	带噪声的 live base / joint / action 特征
History	1570	10 步历史 × 157 维（clean command + proprio）
合计	3007	

Temporal encoders:

- **Motion CNN**: 对 21×64 的 command stack 编码 → 64 维 latent
- **History CNN**: 对 10×157 的 history stack 编码 → 128 维 latent

21-step command block 包含 **current + 20 future** reference frames，这让 teacher 能“提前看到”参考轨迹，从而更稳定地跟踪 motion。

代码对应：

- Teacher 网络: `src/humanoid_hw6/rl/models/teacher.py`
- Teacher 环境: `src/humanoid_hw6/teacher_env_cfg.py`

4.3 可选：回放 Teacher

建议在开始 student 训练前，先用 `play` 观察 teacher 的 tracking 效果：

代码块

```
1 uv run play Mjlab-Humanoid-HW6-Teacher-G1 \  
2   --checkpoint-file checkpoints/g1_hw6_teacher/model_latest.pt \  
3   --motion-file src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml
```

5. 学生策略与 Teacher-Student Distillation

5.1 Student 与 Teacher 的关键差异

Student actor 使用的是 **更接近部署场景** 的 observation：

- 只能看到 **当前参考帧**（而非 teacher 的 21 步 future command block）
- 仍可使用 proprioception 与 history encoder
- 通过 PPO 的 tracking reward 学习跟踪任务

Student 环境已经预先构建了两套 observation group：

- `actor` / `critic`：student 自己的观测
- `teacher_policy`：frozen teacher 的 privileged 观测

代码对应：

- Student 网络: `src/humanoid_hw6/rl/models/student.py`、`src/humanoid_hw6/rl/models/student_teacher.py`
- Student 环境: `src/humanoid_hw6/student_env_cfg.py`

5.2 DistillationPPO 训练框架

本作业在标准 PPO 之上增加一项 distillation loss。总 loss 为：

代码块

```
1 loss = surrogate_loss  
2     + value_loss_coef * value_loss  
3     - entropy_coef * entropy
```

```
4 + distill_coef * distill_loss
```

其中：

- `surrogate_loss`、`value_loss`、`entropy`：标准 PPO 项
- `distill_loss`：你实现的 teacher-student 匹配损失
- `distill_coef`：随训练逐步退火的蒸馏系数

`StudentTeacherActor` 内部持有一个 **frozen teacher**；训练时 teacher 只提供监督信号，**不参与梯度更新**。

代码对应：

- 共享 PPO 循环：`src/humanoid_hw6/rl/algorithms/distillation_ppo.py`
- **请勿修改** 该文件中的 shared PPO loop

5.3 两种 Distillation 方法

本作业要求你实现并对比以下两种方法：

Action Matching

- **目标**：让 student 输出的 action 接近 teacher 在同一 batch 上给出的 action
- **损失**：MSE 或 Huber regression loss
- **直觉**：复制“老师此刻会怎么做”
- **特点**：实现简单、训练稳定、适合作为 baseline

KL Matching

- **目标**：让 student 的 action **distribution** 接近 teacher 的 distribution
- **损失**：mean `KL(teacher || student)`，基于 Gaussian 分布参数
- **直觉**：复制“老师对动作的完整概率分布”，包括 mean 与 std
- **特点**：信息更丰富，可传递 teacher 的不确定性结构

蒸馏系数退火

两种方法都使用 **线性退火** 的 distillation 系数：

- **训练早期**：较大系数，让 student 快速模仿 teacher
- **训练后期**：较小系数，让 PPO reward 信号主导，使 student 适应自身 observation 下的 tracking 任务

以 KL matching 为例，配置中 `kl_coef=0.1` → `kl_coef_min=0.01`，在

`kl_coef_anneal_iters=60_000` 次 update 内线性下降。Action matching 使用类似的

`bc_coef_start` → `bc_coef_end` 机制。

注意：KL matching 中的 distillation KL，与 PPO 内部 adaptive desired_kl 学习率控制器 不是同一个概念。前者是显式的 teacher-student 匹配 loss；后者用于调节 policy update 步长。

6. 代码任务（7 个 TODO）

你需要在 **3 个文件** 中完成 **7 个 TODO**。建议顺序：先完成 `distillation_utils.py` 中的数学工具（TODO 1-5），再完成两种算法的集成（TODO 6-7）。

6.1 代码文件目录索引

文件	用途
<code>README.md</code>	快速上手、命令、teacher observation 摘要
<code>ASSIGNMENT.md</code>	完整作业说明（含 § 3 环境配置与 motion 预览）
<code>src/humanoid_hw6/scripts/data/visualize_motion_curate_viser.py</code>	Viser 浏览 25 个参考 motion clips
<code>src/humanoid_hw6/teacher_env_cfg.py</code>	Teacher 环境配置
<code>src/humanoid_hw6/student_env_cfg.py</code>	Student 环境配置
<code>src/humanoid_hw6/rl/models/teacher.py</code>	Teacher 网络
<code>src/humanoid_hw6/rl/models/student_teacher.py</code>	Student + frozen teacher
<code>src/humanoid_hw6/rl/algorithms/distillation_ppo.py</code>	共享 PPO 循环（只读）
<code>src/humanoid_hw6/rl/algorithms/distillation_utils.py</code>	TODO 1-5：共享数学工具
<code>src/humanoid_hw6/rl/algorithms/action_matching.py</code>	TODO 6：Action matching 集成
<code>src/humanoid_hw6/rl/algorithms/kl_matching.py</code>	TODO 7：KL matching 集成
<code>src/humanoid_hw6/config/g1/rl_cfg.py</code>	Teacher / student RL 配置

6.2 如何快速定位 TODO

在仓库根目录执行：

代码块

```
1  grep -rn "【作业 TODO" src/humanoid_hw6/rl/algorithms/
2  grep -rn "HOMEWORK_TODO" src/humanoid_hw6/rl/algorithms/
```

每个 TODO 使用 `>>> HOMEWORK_TODO_N_START / <<< HOMEWORK_TODO_N_END` 标记边界；函数上方有中文实现提示。删除 `raise NotImplementedError` 后填入你的代码。

文件	TODO 编号
distillation_utils.py	1-5
action_matching.py	6（含 6A 系数退火 + 6B 蒸馏集成）
kl_matching.py	7（含 7A 系数退火 + 7B 蒸馏集成）

6.3 常见错误

现象	可能原因	排查方向
NotImplementedError	TODO 未实现	检查 3 个 algorithm 文件
Teacher weights are not loaded	未提供 teacher checkpoint	确认默认路径或 <code>--agent.teacher-checkpoint-file</code>
Teacher 参与了梯度	未使用 <code>torch.no_grad()</code>	检查 distillation 集成
训练无诊断指标	未返回 <code>DistillationOutput.metrics</code>	检查 action / Gaussian metrics

7. 训练、验证与报告

环境安装与 motion 预览见 § 3 环境配置。本节从测试、训练到实验报告。

7.1 运行测试

代码块

```
1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run python -m pytest
```

若系统安装了 ROS pytest 插件并报 hook 错误，请保留

`PYTEST_DISABLE_PLUGIN_AUTOLOAD=1` 前缀。

建议顺序：先完成 TODO 1（`linear_anneal`），再完成 TODO 2-5，最后完成 TODO 6-7。

7.2 回放 Teacher（可选）

代码块

```
1 uv run play Mjlab-Humanoid-HW6-Teacher-G1 \  
2 --checkpoint-file checkpoints/g1_hw6_teacher/model_latest.pt \  
3 --motion-file src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \  
4 --num-envs 1 \  
5 --viewer viser
```

7.3 训练 Student

默认 logger 是 `wandb`。本地开发建议关闭 W&B 并使用 TensorBoard：

Action matching（完整训练示例）：

代码块

```
1 WANDB_MODE=disabled uv run python -m mjlab.scripts.train \  
2 Mjlab-Humanoid-HW6-Student-Action-Matching-G1 \  
3 --agent.logger tensorboard \  
4 --env.scene.num-envs 4096 \  
5 --env.commands.motion.motion-file \  
src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \  
6 --agent.teacher-checkpoint-file checkpoints/g1_hw6_teacher/model_latest.pt \  
7 --agent.run-name action_matching_run
```

KL matching（完整训练示例）：

代码块

```
1 WANDB_MODE=disabled uv run python -m mjlab.scripts.train \  
2 Mjlab-Humanoid-HW6-Student-KL-Matching-G1 \  
3 --agent.logger tensorboard \  
4 --env.scene.num-envs 4096 \  
5
```

```

5  --env.commands.motion.motion-file
   src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \
6  --agent.teacher-checkpoint-file checkpoints/g1_hw6_teacher/model_latest.pt \
7  --agent.run-name kl_matching_run

```

Smoke test（快速验证能否训练）：

代码块

```

1  WANDB_MODE=disabled uv run python -m mujlab.scripts.train \
2  Mujlab-Humanoid-HW6-Student-Action-Matching-G1 \
3  --agent.logger tensorboard \
4  --env.scene.num-envs 4096 \
5  --agent.max-iterations 5000 \
6  --env.commands.motion.motion-file
   src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \
7  --agent.run-name smoke_action_matching

```

训练日志目录：

代码块

```

1  logs/rsl_rl/g1_hw6_student_action_matching/<timestamp>_<run_name>/
2  logs/rsl_rl/g1_hw6_student_kl_matching/<timestamp>_<run_name>/

```

每个 run 会保存 `model_latest.pt`、`params/`、`motion_stats/motion_stats_latest.csv` 等文件。

7.4 TensorBoard

代码块

```

1  tensorboard --logdir logs/rsl_rl/g1_hw6_student_action_matching
2  # 或
3  tensorboard --logdir logs/rsl_rl/g1_hw6_student_kl_matching

```

建议重点观察：

指标	含义
<code>Train/mean_reward</code>	总回报
<code>Train/mean_episode_length</code>	episode 长度

Termination_Frac/fail	失败比例
Termination_Frac/time_out	超时比例
Loss/distill_coef	蒸馏系数退火
Loss/bc / Loss/kl	蒸馏主 loss
Loss/action_mae / Loss/action_rmse	Action matching 诊断
Loss/mean_rmse / Loss/std_rmse	KL matching 诊断

`motion_stats/motion_stats_latest.csv` 记录训练过程中各 clip 的 `failure_ema`、`completion_ema` 等统计，可用于比较两种算法在不同 motion clip 上的表现。

7.5 Play 验证

训练完成后，用 **与训练时相同的 task ID** 回放 student checkpoint:

代码块

```
1 uv run play Mjlab-Humanoid-HW6-Student-Action-Matching-G1 \
2   --checkpoint-file logs/rsl_rl/g1_hw6_student_action_matching/2026-07-11_17-
45-41_smoke_action_matching/model_latest.pt \
3   --motion-file src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \
4   --num-envs 1 \
5   --viewer viser
```

代码块

```
1 uv run play Mjlab-Humanoid-HW6-Student-KL-Matching-G1 \
2   --checkpoint-file logs/rsl_rl/g1_hw6_student_kl_matching/2026-07-11_17-57-
22_smoke_kl_matching/model_latest.pt \
3   --motion-file src/humanoid_hw6/config/g1/motion_data_cfg_g1_accad_walk.yaml \
4   --num-envs 1 \
5   --viewer viser
```

请将路径中的 `<timestamp>_<run_name>` 替换为你自己的 run 目录。

7.6 评估协议（实验报告用）

在 **相同 25 clips、相近训练步数** 下，对比 action matching 与 KL matching:

1. **定量**: TensorBoard 曲线 (reward、episode length、fail/timeout、distill loss、诊断指标)
2. **Per-clip**: `motion_stats_latest.csv` 中的 completion / failure 统计

- 3. **定性**: Viser 中观察 tracking 稳定性、是否频繁倒地、步态自然度
- 4. **分析**: 诊断指标是否与视觉表现一致; 哪种方法更好; 各自适用场景

7.7 实验报告建议

报告建议包含:

1. **Teacher-student distillation 原理简述**
 2. **Action matching vs KL matching 对比**
 - 训练稳定性 (loss 曲线、`distill_coef`、诊断指标)
 - Play 中的 tracking 质量 (有哪些tracking相关指标? 请说出你的理解)
-

8. 评分检查清单

- ☐ 7 个 TODO 全部实现, uv run pytest 通过
 - ☐ Action-matching student 可正常训练, 无 teacher-load 错误
 - ☐ KL-matching student 可正常训练, 无 teacher-load 错误
 - ☐ TensorBoard / Viser 中完成两种方法的对比
 - ☐ 提交简要实验报告, 包含 action vs KL 行为对比(tensorboard日志或者wandb日志)
 - ☐ 压缩包命名符合 `用户名-实践6.zip` 规范
-

9. 提示 Hints

- 用 `grep -rn "【作业 TODO" src/humanoid_hw6/rl/algorithms/` 快速跳转到每个 TODO。
 - 建议顺序: TODO 1 → 2 → 3 → 4 → 5 → 6 → 7。
 - Teacher 权重保持 frozen; 只优化 student actor。
 - KL matching 必须使用你自己实现的 `diagonal_gaussian_kl`, 不要与 PPO 的 `desired_kl` 混淆。
 - `DistillationOutput.metrics` 会自动进入 TensorBoard (键名如 `Loss/action_mae`)。
 - Smoke test 时可减少 `--agent.max-iterations`; 完整对比建议使用相近训练步数。
-

10. 提交说明

10.1 提交方式与截止时间

- **提交渠道：** 深蓝学院课程平台作业入口（以课程通知为准）
- **截止时间：** 以课程平台显示的 HW6 截止日期为准；逾期规则以平台/policy 为准
- **提交次数：** 通常以 **最后一次有效提交** 为准；具体以平台设置为准

10.2 提交内容

请打包为 **单个 .zip 文件**，大小在100M以内，包含以下两部分：

(1) 代码：code/ 目录

只需提交你修改过的 **3 个源文件**，保持相对路径不变：

代码块

```
1  code/
2  └── src/humanoid_hw6/rl/algorithms/
3      ├── distillation_utils.py    # TODO 1-5
4      ├── action_matching.py      # TODO 6
5      └── kl_matching.py           # TODO 7
```

(2) 报告：report/ 目录

代码块

```
1  report/
2  ├── 用户名-实践6.pdf            # 必须；若用 Markdown 则同时提供 PDF
3  └── figures/                    # TensorBoard 训练日志
4      ├── tb_action_matching.png  # Viser 演示视频
5      ├── tb_kl_matching.png
6      └── viser_*.mp4/gif
```

11. 延伸阅读（References）

1. Yin, S., Ze, Y., Yu, H.-X., Liu, C. K., & Wu, J. **VisualMimic: Visual Humanoid Loco-Manipulation via Motion Tracking and Generation**. arXiv:2509.20322, 2025.

<https://arxiv.org/abs/2509.20322>

2. Wang, Z., Li, Y., Ma, T., Zhang, Q., Fan, Y., Xu, H., Yang, S., & Liang, J. **Perceptive Behavior Foundation Model: Adapting Human Motion Priors to Robot-Centric Terrain**. arXiv:2606.08059, 2026.

<https://arxiv.org/abs/2606.08059>

3. Ze, Y., Zhao, S., Wang, W., Kanazawa, A., Duan, R., Abbeel, P., Shi, G., Wu, J., & Liu, C. K. **TWIST2: Scalable, Portable, and Holistic Humanoid Data Collection System.** arXiv:2511.02832, 2025.

<https://arxiv.org/html/2511.02832v1>

4. Mahmood, N., Ghorbani, N., Troje, N. F., Pons-Moll, G., & Black, M. J. **AMASS: Archive of Motion Capture as Surface Shapes.** ICCV, 2019.

<https://amass.is.tue.mpg.de/>
